

Welcome to ECE452

LECTURE 1: Introduction

Part A: About the Course

Zhao Zhang
Rutgers University

Course Information

❑ Lectures: *Monday* 8:30 AM – 9:50 AM & *Thursday* 8:30 AM – 9:50 AM

- Classroom: EN-B120
- Canvas sites: <https://rutgers.instructure.com/courses/385187>
- Course format: **In-person class**. See attendance and participation for more policies.
- Instructor Office Hours: 1-2PM Wednesday
- Instructor Office: CoRE 506

❑ Class Announcement

- We will use Canvas for major course-related announcements

❑ Teaching Assistant Office hours:

- Haoshen Yang :
 - **TBD**
 - Location: In person: TBD
- Xin Li:
 - **TBD**
 - Location: In person: TBD

Team Projects

- ❑ Grading is mostly project-based:
 - **65%** of course grade is based on team projects
 - **25%** from individual homework assignments
 - **10%** of popup quizzes and in-class participation
- ❑ There will be NO exams
 - NO exams in this course
- ❑ Create teams of 6 students
 - <https://forms.gle/Q3NH4WzaGKM9rBHB9>
- ❑ **DEADLINE** : Thursday, **Feb 5th, 2025** (before midnight)
After that, teams assigned randomly

Course Goals and Structure

- ❑ Text Book:
<https://drive.google.com/file/d/19dqWms9qUQ2Bi8JfsBhC5Q6zDeJd0s5o/view>
- ❑ This course will cover basic concepts of software engineering
 - Agile Methods
 - Test Driven Design
 - Software architecture with emphasis on REST and microservices
 - Testing methods
 - Advanced topics in JavaScript programming
 - Students are expected to be familiar with the basics of JavaScript and Python and we will not cover the basics of JavaScript in the class
- ❑ Goal: To build a Web Service for Agentic System (similar to ChatGPT)
 - Team projects are organized in three iterations, the details of which will be specified
- ❑ All students are required to use only the fundamental web technologies: JavaScript, HTML, CSS, and Python
 - As for the libraries or frameworks, nothing other than NodeJS to run JavaScript on the server side, an SQL-based database (example: MySQL) or NoSQL database (example: Firebase), vLLM as the LLM serving platform.

Course Project

- ❑ Each team has 6 students
- ❑ The goal is to build a web service for an agentic system
 - You can think about it as ChatGPT or Gemini
 - There is a large language model (LLM) in the backend
 - There is a web user interface in the frontend
 - Users input prompts and get generated texts from LLM
- ❑ The project consists three iterations
 - Iteration 1: Authentication, Feb 26 – Mar 16
 - Iteration 2: TBD, Mar 17 – Apr 9
 - Iteration 3: TBD, Apr 10 – Apr 30
- ❑ There are three roles for each iteration, students will rotate their roles across iterations
 - Requirement Engineers: Requirement analysis
 - Test Engineers: Test Design and Acceptance
 - Implementation Engineers: Development, Unit Tests

Course Project

- ❑ Teams are required to use GitHub for collaboration
 - Use GitHub projects to manage tasks
 - Use GitHub issues to resolve bugs
- ❑ Some other tools
 - Cucumber.js to convert user stories to acceptance tests
 - Jasmine for unit tests
 - vLLM as the LLM backend
 - Express.js as the backend MVC framework
 - Flake8 for Python style and ESLint for JavaScript style checking
- ❑ Submission
 - GitHub Repo
 - Projects
 - Issues
 - Commits
 - Reports – one report per iteration
 - Requirements
 - Tests
 - Design

Course Project – Just in Case

- ❑ What if your team member does not respond to your email within 48 hours?
 - Give him/her a second chance
 - If his/her task is blocking your progress, you can request the backup solution from the TAs
 - Gradings will be done individually, so if others do not finish their task, they don't impact your grades

Homeworks

- ❑ Two coding homeworks
- ❑ Submission guideline
 - Source code – with unit tests
 - Documentation
 - Submitting via Canvas

Welcome to ECE452

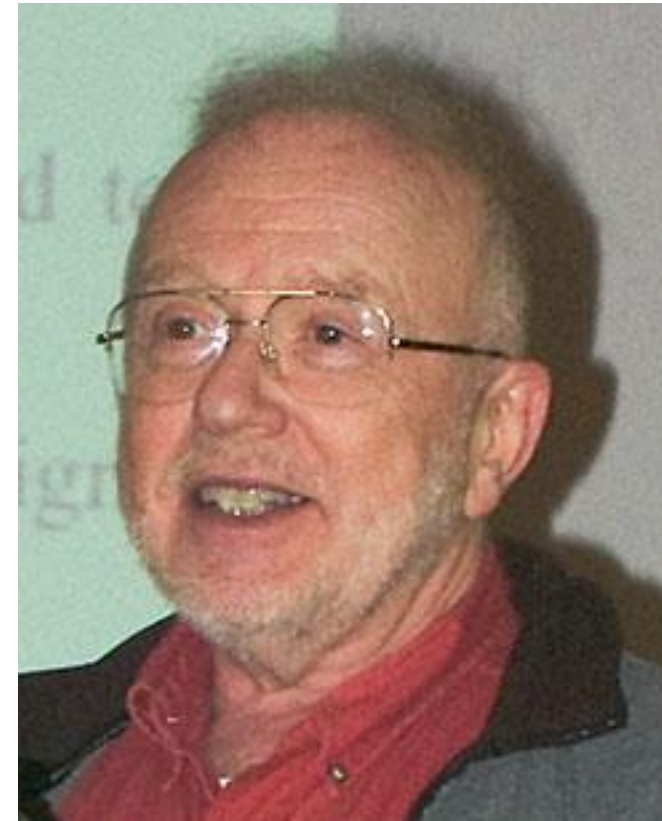
LECTURE 1: Introduction

Part B: Software Engineering

Zhao Zhang
Rutgers University

“[SW engineering is]
multi-person
development of
multi-version
programs.”

David L. Parnas, software engineering pioneer
noted for contributions to modular design
(information hiding), software safety, and precision
in technical documentation



What makes a great SW Engineer?

Personal characteristics	Team skills
Always improving Passionate Technically open-minded Data-driven	Helps others understand by tailoring explanations to them Creates shared success possibly via personal compromises Creates “safe haven” where others can learn from mistakes Gives honest feedback
Decision Making	Technical Skills
Knows people & organization Updates mental models when learn new skills/facts/context Consider situation at multiple levels when making judgments Can reason about complex & intertwining ideas	Elegant solutions Creative thinking when faced with limitations of current solutions Anticipates technical needs based on prior experience

From *What Makes a Great Software Engineer?*

P. Li, A. Ko, J. Zhu, Microsoft & Univ. of Washington, Proc. ICSE 2015, 59 interviews with SWE experts at Microsoft

Top 10 ways to have a bad time

- 10. Jump right into coding—skip the “passive learning” part
- 9. Don't read the textbook
- 8. Work alone, don't help peers
- 7. Resist learning new tools/techniques
- 6. Ignore course staff advice
- 5. Focus on getting points vs. learning
- 4. Learn exclusively from videos
- 3. Ignore the Honor Code
- 2. Multitask while coding
- 1. Don't waste valuable coding time on “processes”

Fully remote programming?



- <https://overcast.fm/+Ys-5mjY2Q>

Evidence: programmers get *less feedback* when remote vs. in person

➔ work with colleagues when you can!

When asking questions ...

- Search before you ask.
- Be civil and professional. Write as if your coworkers/supervisor are reading it (because in a way they are).
- Help us help you by *being specific*.
 - Not useful: “My code doesn’t work. Here is a dump of my code and 50 lines of backtrace. Help me!” *No one but you should be determining what to type.*
 - Useful: “I specifically expected the below code to do X, but instead it does Y. I tried searching for the error message, checking StackOverflow, etc. but I found nothing relevant. The first place in the backtrace that my code is implicated is [line N]. Any idea what I’m doing wrong?”

Early Software Development Processes: Plan-and-Document

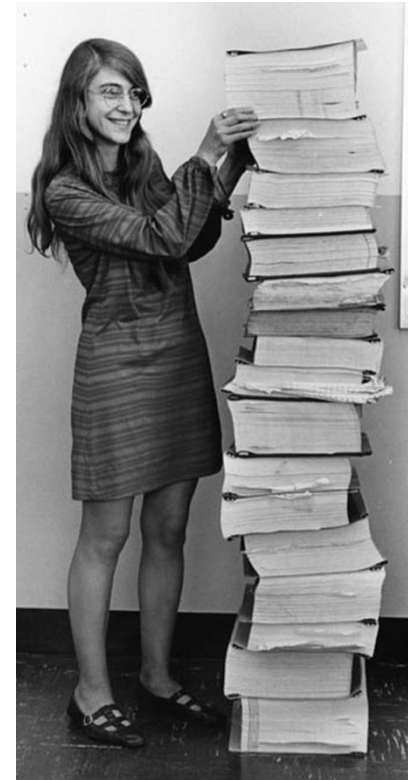
ESaaS §1.2

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



“Software crisis” leads to SW Eng.

- “Software engineering” term popularized by inaugural 1968 conference, sponsored by NATO
- “Software crisis”: SW development was not pacing increasing power of hardware & problem complexity
- USA’s Apollo (manned moon mission) control software was a highly visible example
- Q: Can we build software as predictably and reliably as we build bridges & buildings?



“Plan-and-Document”

- Before development, project manager makes plan
- Write detailed documentation for all plan phases
- Progress measured against plan
- Changes to project must be reflected in documentation & plan

First P&D process: Waterfall (1970)

1. Requirements analysis & specification
2. Architectural design
3. Implementation & integration
4. Verification
5. Operations & maintenance

Phases can take months; document everything since team can change in that time.

How well does Waterfall work?

- And the users exclaimed with a laugh and a taunt: “It’s just what we asked for, but not what we want.” —Anon.
 - Often when you build what you planned, you find it doesn’t work
- “Plan to throw it away and build it right” —Ken Thompson (1999 Turing Award)
- Often after the fact, they realize they should have built it differently
- “Adding manpower to a late software project makes it later” —Brooks’s Law
- Led to Spiral, RUP, and other processes

Unlike hardware, software must be able to deal with change.

Fred Brooks Jr.
1931-2022



Software Development Processes: Agile

ESaaS §1.3

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



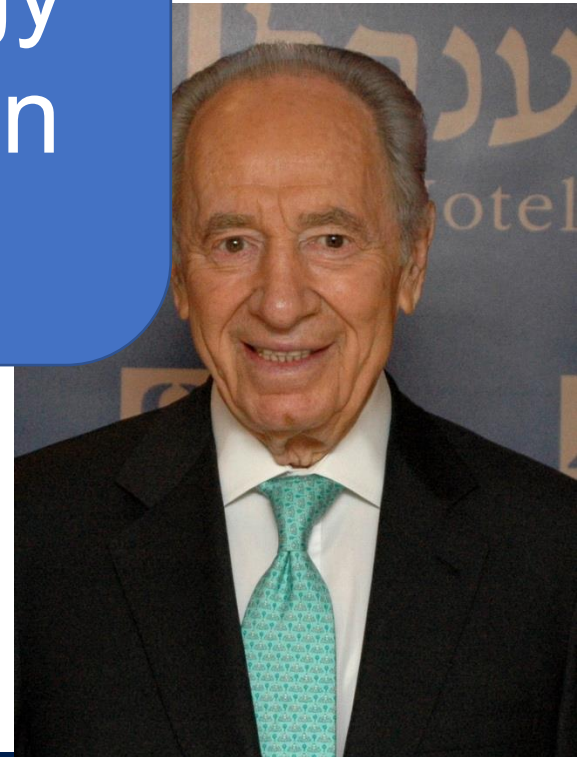
“Peres’s Law”

“If a problem has no solution,
it may not be a problem
not to be solved.”

— Shimon

Software change is a fact of
life. We need a methodology
that copes with it rather than
trying to “solve” it.

(former Prime Minister
winner of 1994 Nobel Peace Prize
for Oslo Accords)



Agile Manifesto, 2001

“We are uncovering better ways of developing SW by doing it and helping others do it. Through this work we have come to value

- Individuals and interactions over processes & tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.” —Kent Beck et al.

Agile vs. Plan-and-Document

- Embraces change as a fact of life: continuous improvement vs. phases
- Developers continuously refine working but incomplete prototype until customers happy, with customer feedback on each **Iteration** (every ~1 to 2 weeks)
- Uses lighter-weight mechanisms, such as *user stories* rather than detailed requirements elicitation
- Controversial in 2001, used by most SW co.'s by 2012

Extreme Programming (XP)

- If short iterations with customer are good, make them as short as possible (weeks vs. years)
- If simplicity is good, always do the simplest thing that could possibly work
- If testing is good, test all the time. In fact, write the test before you write the code, then write the code to pass the test.
- If code reviews are good, review code continuously: program in pairs, taking turns looking over each other's shoulders.
 - and turn every proposed code change into a mini-code-review (pull request)

Agile isn't always best!

Yes:
P&D

No:
Agile

1. Is specification required?
2. Are customers unavailable?
3. Is the system to be built large?
4. Is the system to be built complex (e.g., real time)?
5. Will it have a long product lifetime?
6. Are you using poor software tools?
7. Is the project team geographically distributed?
8. Is team part of a documentation-oriented culture?
9. Does the team have poor programming skills?
10. Is the system to be built subject to regulation?

Testing and Software Quality

ESaaS §1.4

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



What is *software quality*?

- Product quality (in general): “fitness for use”
 - Business value for customer *and* manufacturer
 - *Quality Assurance* : processes/standards to make it happen
- Software quality:
 1. Satisfies customers’ needs—easy to use, gets correct answers, does not crash, ...
 2. Be easy for developer to debug and enhance
- Software QA: ensure quality and improve processes in SW organization
- *In modern Agile development, QA is everyone’s responsibility, not that of a separate QA team*

Assurance: Verification/Validation

- Verification: Did you build *the thing right*?
 - Did you meet the specification?
- Validation: Did you build *the right thing*?
 - Is this what the customer wants?
 - Is the specification correct?
- Hardware focus generally *verification*
- Software focus generally *validation*
- 2 options: Testing and Formal Methods

Exhaustive testing is infeasible

- Divide and conquer: perform different tests at different phases of SW development
 - Higher levels don't repeat tests of lower levels
- *Coverage*: various measurements of what % of system is “exercised” by test suite

System or acceptance test:
integrated program meets its specifications

Integration test: interfaces between units have consistent assumptions, communicate correctly

Module or functional test:
across individual units

Unit test: single method does what was expected

Productivity: Conciseness, Synthesis, Reuse, and Tools

ESaaS §1.5

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Why Productivity Matters

- 50 years of Moore's Law (~2x every 1.5 years)
 - ⇒ HW got faster and more powerful
 - ⇒ SW designs got more complicated
 - ⇒ Had to improve SW productivity
- Historically, 4 main techniques
 1. Clarity via conciseness
 2. Synthesis
 3. Reuse
 4. Automation and Tools

1/4: Clarity via conciseness

1. Raise the level of abstraction:

- HLL programming languages vs. assembly lang
- Automatic memory management (Java vs. C)
- Reflection, metaprogramming

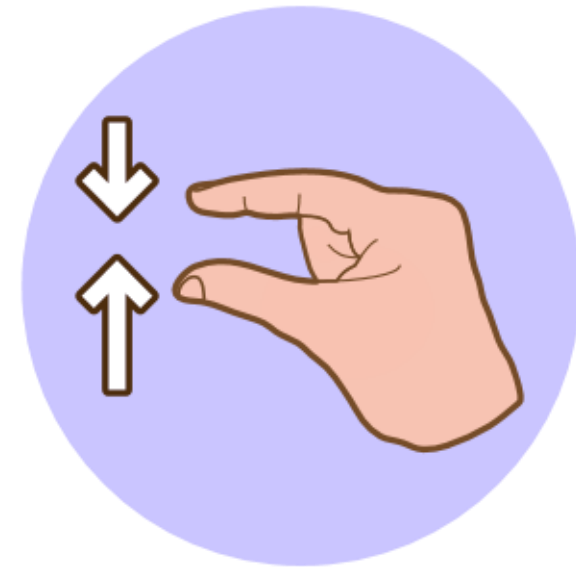
2. Exploit VHLL flexibility to get more concise syntax

`assert_greater_than_or_equal_to(a, 7)`

vs. `expect(a).to be ≥ 7`

`when = Time.now - 2*60*60`

vs. `when = 2.hours.ago`

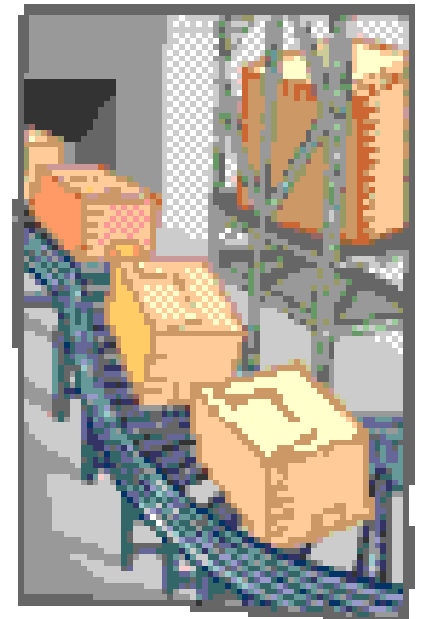
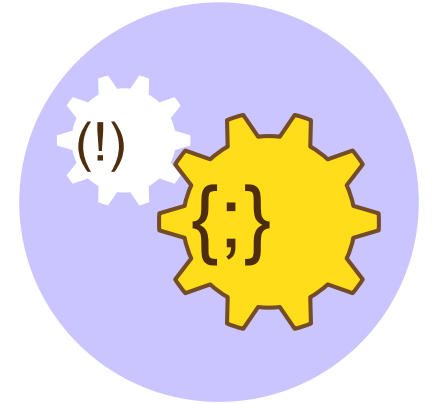


```
if (index == 0)
    str = "A"
elsif (index == 1)
    str = "B"
elsif (index == 2)
    str = "C"
elsif (index == 3)
    str = "D"
elsif (index == 4)
    str = "E"
else
    str = ""
end
```

```
str = if index.between?(0,4) then 'ABCDE'[index]
      else ''
```

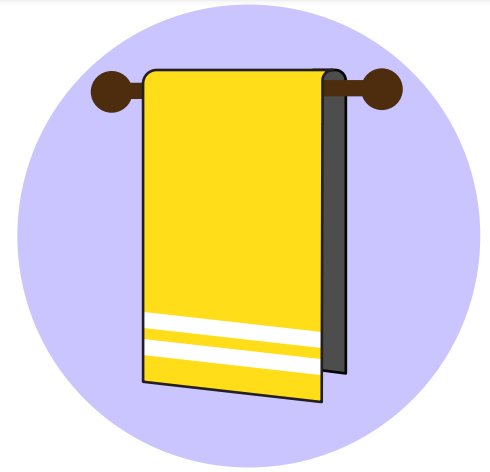
2/4: Synthesis—code that writes code

- BitBlit (c. 1975): self-modifying code to remove conditional test in inner loop
- Common uses today: templating and code generators



3/4: Reuse—DRY it out!

- Don't Repeat Yourself (DRY)
 - *Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.*
—Andy Hunt & Dave Thomas, 1999
 - vs. apply same repair in many places manually, copy-paste-tweak code, etc.
- How? Identify commonalities & refactor code to exploit them

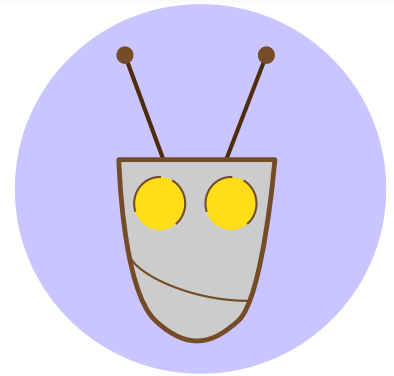


3/4 DRY, continued

1. Procedures and functions
2. Standardized libraries (reuse single task)
3. Object oriented programming: reuse and manage *collections* of tasks
4. Design patterns: reuse a general strategy even if implementation varies
5. (large language models: reuse prompts?)

4/4: Automation vs. manual task

- Saves time, improves accuracy
 - (e.g., make)
- New tool desiderata: learning curve justifies productivity gained
- **SWE Life skill:** learn to use new tools
- (Lots of chances in this course: Cucumber, Jasmine, GitHub, ...)



Deploying SaaS (Software as a Service): SOA (Service-Oriented Architecture), and Utility (Cloud) Computing

ESaaS §1.6–1.7



© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



SaaS > SWS (shrink-wrapped software)!

- No install/upgrade worries about HW capability, OS, etc.
- Data stored safely, persistently on servers
- Easy group access to data for collaboration
- If data is large or changed frequently, simpler to keep 1 copy at central site
- Upgrades apply to all users immediately; beta-test new features on 1% of users

Even traditional “productivity” apps (word processing, financial planning, spreadsheets) now SaaS

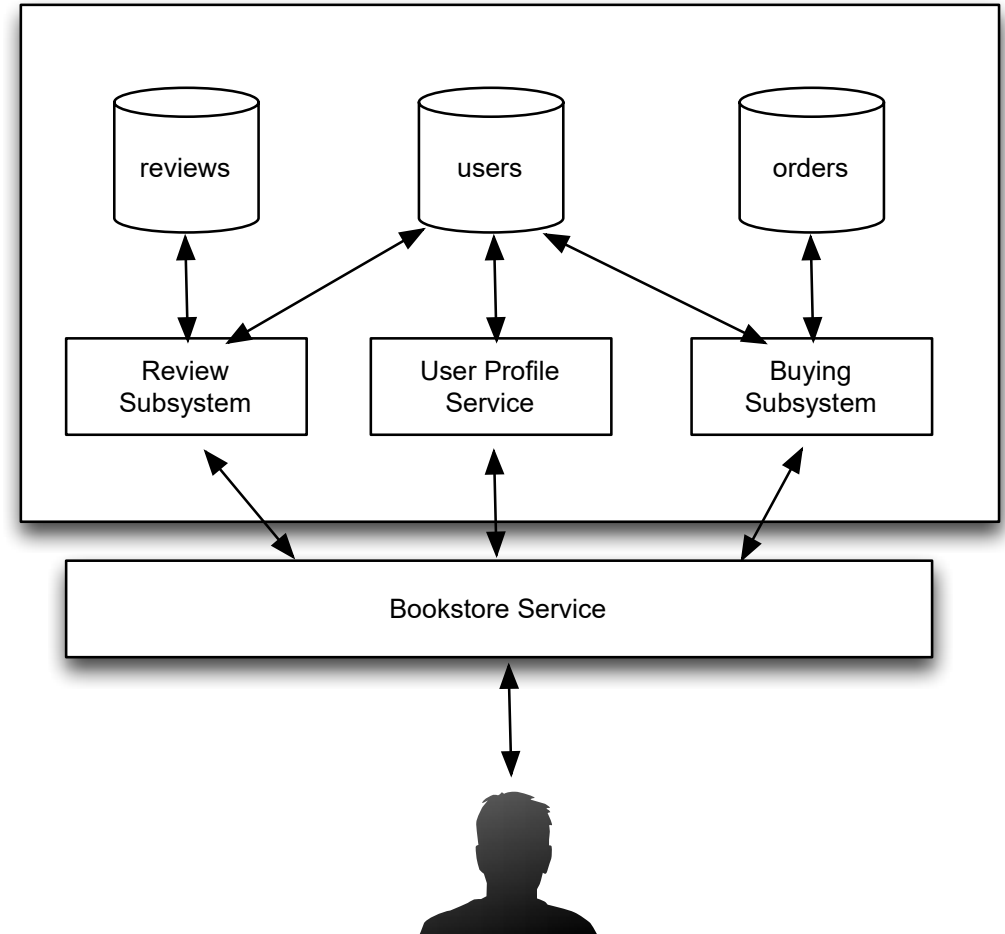
SOA: how to DRY out SaaS!

“[Amazon CEO Jeff Bezos] realized long before the vast majority of Amazonians that Amazon needs to be a platform.”
Steve Yegge, former Googler, former Amazon engineer, in a 2011 blog post



Online bookstore as plain SaaS

- Internal subsystems can share data directly
 - Review access user profile
- All subsystems inside single API (“Bookstore”)



(Figure 1.2, *Engineering Software as a Service* by Armando Fox and David Patterson, 2nd Beta edition, 2013.)

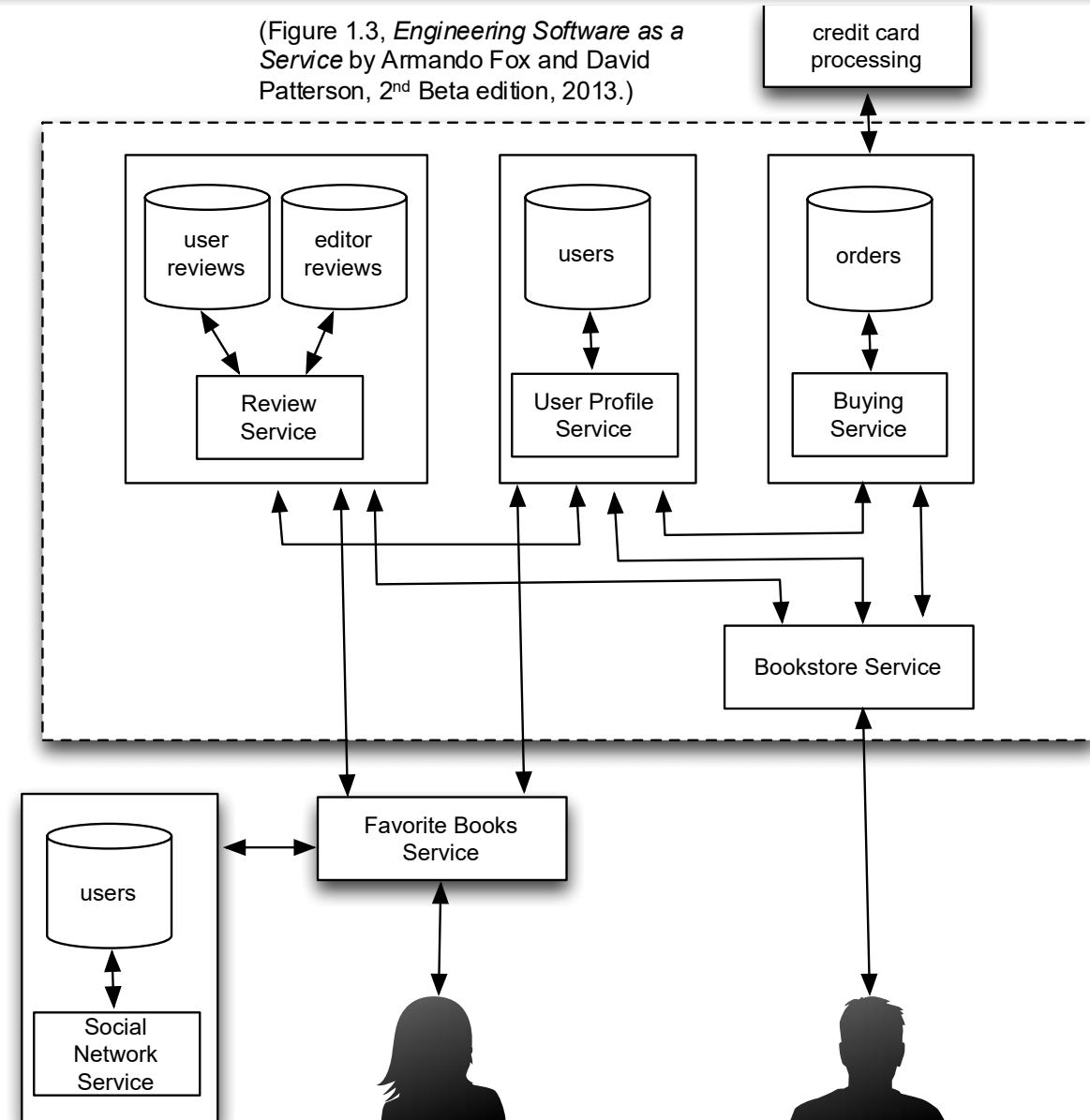
Online bookstore as SOA

- Subsystems independent, as if in separate datacenters
 - Review Service accesses User Service API
- Can recombine to make new service (“Favorite Books”)



Make it work like this or you're fired.
(Jeff Bezos in 2002)

(Figure 1.3, *Engineering Software as a Service* by Armando Fox and David Patterson, 2nd Beta edition, 2013.)



Ideal hardware for SaaS?

- SaaS makes 3 demands on infrastructure
 1. Communication
 2. Scalability
 3. Dependability

Clusters & Warehouse-scale computers

- late 1990s: supercomputers → late 2000s: datacenters of 100K's of commodity PCs ("warehouse scale")
 - Economics of scale in hardware and operations pushed down cost of largest data
 - Scal
 - Dep
 - supp
- Utility c
Microsoft Azure...)
 - Traditional private datacenters were utilized 10% - 20%, so consolidate many 100s of customers into a few warehouse-scale datacenters
 - illusion of infinite scalability to multiple customers ("public cloud"):
1000 computers @ 1 hour cost same as 1 computer @ 1000 hours

**Eliminates capital costs
for SaaS startups**

Deploying SaaS: Browsers & Mobile

ESaaS §1.8

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



Smartphones & mobile apps

- 2007: Apple iPhone
- 2018: >50% web traffic from mobile¹
- 2020: ~50% smartphone penetration²



Apps should be either
“Mobile-friendly” or “Mobile-first”

- Responsive design & accessibility (a11y)
 - Thinking about interaction
- ➔ *What technologies can help us?*

[1] <https://www.perficientdigital.com/insights/our-research/mobile-vs-desktop-usage-study>

[2] <https://www.statista.com/statistics/330695/number-of-smartphone-users-worldwide/>

Image: <https://pxhere.com/en/photo/925815>, CC0 public domain

Introduction

This article is a review of the book Dietary Preferences of Penguins, by Alice Jones and Bill Smith. Jones and Smith's controversial work makes three hard-to-swallow claims about penguins:

First, that penguins actually prefer tropical foods such as bananas and pineapple to their traditional diet of fish

Second, that tropical foods give penguins an odor that makes them unattractive to their traditional predators

`<h1>Introduction</h1>`

`<p>`

This article is a review of the book
`<i>`Dietary Preferences of Penguins`</i>`,
by Alice Jones and Bill Smith. Jones and Smith's
controversial work makes three hard-to-swallow claims
about penguins:

`</p>`

`<ul>`

`<li>`

First, that penguins actually prefer tropical foods
such as bananas and pineapple to their traditional diet
of fish

`</li>`

`<li>`

Second, that tropical foods give penguins an odor that
makes them unattractive to their traditional predators

`</li>`

`</ul>`

...

Introduction

This article is a review of the book *Dietary Preferences of Penguins*, by Alice Jones and Bill Smith. Jones and Smith's controversial work makes two hard-to-swallow claims about penguins:

- First, that penguins actually prefer tropical foods such as bananas and pineapple to their traditional diet of fish
- Second, that tropical foods give penguins an odor that makes them unattractive to their traditional predators

...

```
<h1>Introduction</h1>
```

```
<p>
```

```
This article is a review of the book  
<i>Dietary Preferences of Penguins</i>,  
by Alice Jones and Bill Smith. Jones  
and Smith's controversial work makes  
three hard-to-swallow claims about  
penguins:
```

```
<ul>
```

```
<li>
```

```
First, ...
```

HTML in a nutshell

- Document = Hierarchy of *elements*
 - text/inline (headings, tables, lists, paragraphs)
 - embedding other object types (images, JavaScript)
 - forms to submit simple input (text, radio/check buttons, menus...)
- Elements delimited by `< > . . . </ >`
 - Some have *attributes* and/or *content*:
`<p class="strong">Hello world</p>`
 - Some can nest inside others:
`<div> <p class="lead">...</p> </div>`

Cascading stylesheets (CSS) control visual appearance

```
<link rel="stylesheet" href="https://.../styles.css"/>
```

HTML `id` & `class` attributes particularly important in CSS

- *id* must be *unique within this page*
- same *class* can be attached to many elements, even different types

```
<div id="intro" class="text-info">
```

```
<p>
```

This article is a review ...

```
</p>
```

```
</div>
```

*Goal: HTML markup only shows structure,
not visual appearance*

CSS selectors: a life skill

```
<div class="container" id="container">
  <h1>
    Welcome, <span class="name">Armando</span>
  </h1>
  
</div>
```

- tag name: `h1`
- class name: `.container`
- element ID: `#container`
- tag name & class: `div.container`
- descendant relationship: `div .custName`
- Attributes *inherit* browser defaults unless overridden

What's in a CSS file?

```
body {  
  font-family: "Helvetica", "Arial", "sans-serif";  
}  
#content.plain {  
  background-color: white;  
}  
#footer {  
  text-align: center;  
}  
#report_body ul {  
  list-style: disc;  
}  
.validationError {  
  color: red;  
}  
.fieldWithErrors {  
  padding: 2px;  
  color: white;  
  background-color: red;  
}
```

<https://git.io/Jfl9i>

Beautiful Code, Legacy Code, and Main Ideas of Chapter 1

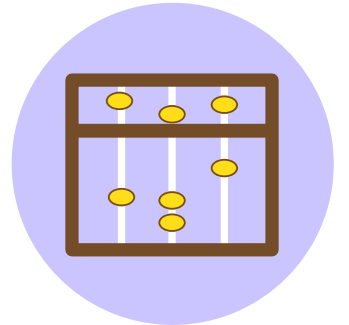
ESaaS §1.9, 1.11, 1.12

© 2023 Armando Fox, David Patterson, Michael Ball
Licensed under Creative Commons Attribution-
NonCommercial-4.0 Unported License



What is legacy code?

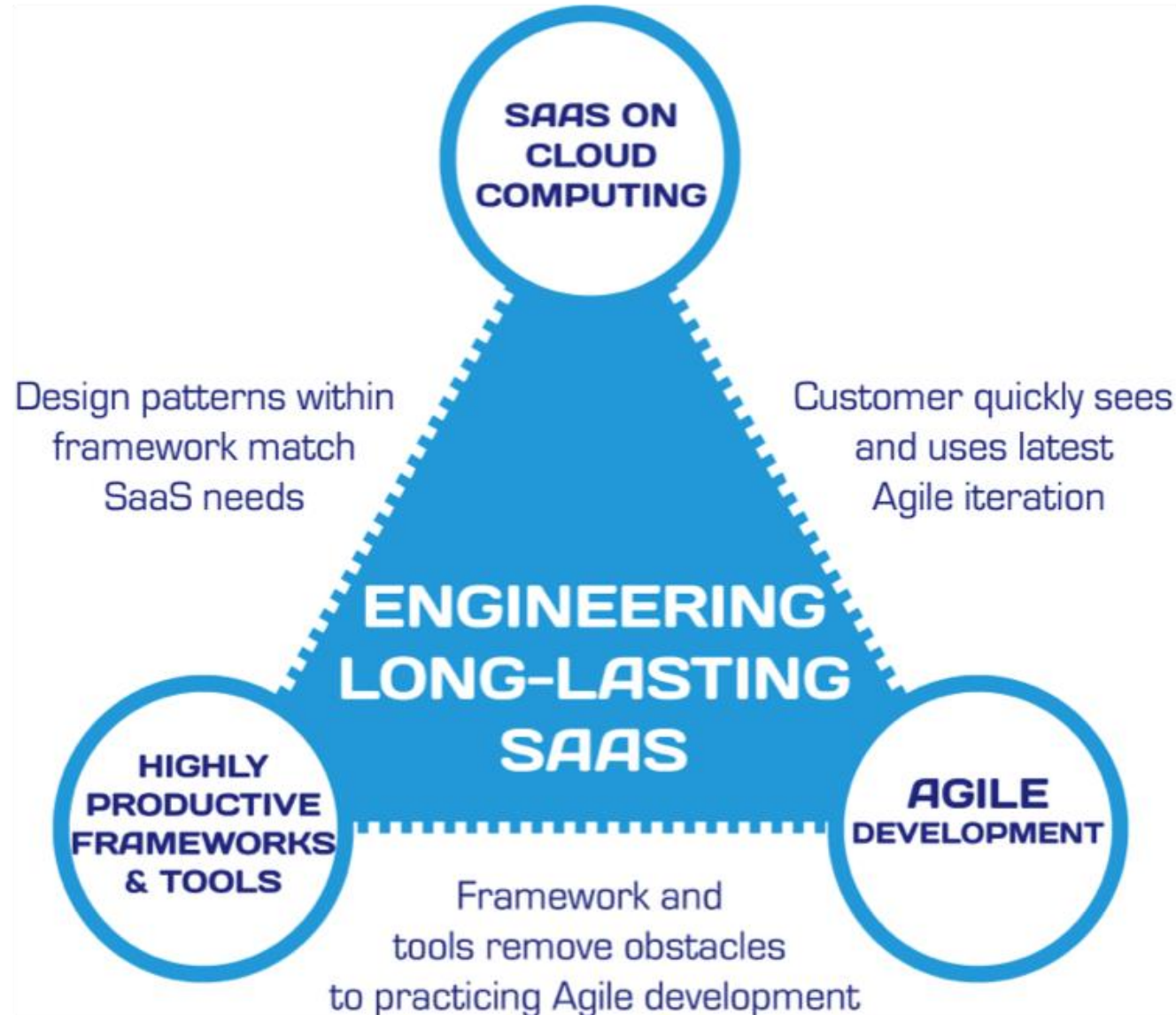
- If SW change is fact of life, SW must be “easy” to evolve
- But older SW that *still serves customer need* may be difficult to evolve due to design inelegance or antiquated technology
 - 60% SW maintenance costs enhancing legacy SW
 - 17% for fixing bugs
 - *Vital but ignored topic in most SWE courses*
- Can Agile help with legacy code?



Fallacy: Agile is the best lifecycle

- Good match for some SW, especially SaaS
 - But not for NASA, code subject to regulations, safety-critical code, ...
- Per topic, will practice Agile way to learn but will also see Plan & Document perspective
 - you will see new lifecycles in response to new opportunities in your career → expect to learn new ones

ESaaS in 2 slides: #1 of 2



ESaaS in 2 slides: #2 of 2

